

Five colors for 1-saturated subcubic graphs with triangular degree-three vertices

Matias Ortiz

5 October 2026

Abstract

We give a candidate computer-assisted proof that every finite simple subcubic graph in which each degree-three vertex belongs to a triangle and has at most one degree-three neighbor has packing chromatic number at most five. The argument reduces the graph class to cycles with separated triangular ears. A finite automaton records the last five colors on the underlying cycle. Eighteen explicitly specified Boolean relations form an inclusion certificate that handles every number of triangles and every permitted sequence of gap lengths. A twelve-vertex graph shows that the bound is sharp.

Draft status. The finite certificate has passed separately implemented checks. This document is for proof review; it has not been independently reviewed by another mathematician, submitted, or peer reviewed. The proposed theorem addresses Conjecture 2 of El Zein and Mortada [1]. A subsequent-paper search found related progress [2], but no resolution of this particular conjecture.

1 Statement and plan

A *packing coloring* of a graph G is a map $f : V(G) \rightarrow \{1, \dots, k\}$ such that distinct vertices receiving color c are at distance strictly greater than c . The least possible k is the packing chromatic number $\chi_\rho(G)$. Distances between different components are infinite. A graph is *subcubic* if its maximum degree is at most three. It is *1-saturated* if each degree-three vertex has at most one degree-three neighbor.

Theorem 1 (proposed). *Let G be a finite simple subcubic graph. Suppose every degree-three vertex belongs to a triangle and has at most one degree-three neighbor. Then $\chi_\rho(G) \leq 5$. The bound is attained.*

The local girth of a vertex is the length of its shortest containing cycle. The definition in [1] is

$$g_3(G) = \max\{g(v) : d_G(v) = 3\}.$$

Thus $g_3(G) = 3$ means that *every* degree-three vertex is triangular. Our formulation also covers components with no degree-three vertices, and does not assume minimum degree two. We use the definition of saturation directly, so vertices of degree one cause no ambiguity.

The proof has three ingredients: a structural reduction, a sufficient coloring construction on the reduced family, and a finite certificate that the construction works for all gap sequences. Only the explicitly stated finite assertions in Section 5 and the sharpness check require computation.

2 Reduction to triangle necklaces

For integers $t \geq 1$ and $g_1, \dots, g_t \geq 2$, define $N(g_1, \dots, g_t)$ by taking disjoint triangles $L_i R_i T_i$ and joining R_i to L_{i+1} with a path of exactly g_i edges. All internal vertices of these paths are new and distinct; indices are taken modulo t . We call these graphs *triangle necklaces*. Deleting the tips T_i leaves a backbone cycle of length

$$L = \sum_{i=1}^t (g_i + 1), \quad |V(N)| = L + t.$$

Its edges $L_i R_i$ are called marked edges. Notice that $N(2)$ is the diamond $K_4 - e$. A gap counts edges, not internal vertices.

Lemma 2 (structural reduction). *Every connected graph satisfying the hypotheses of Theorem 1 is either two triangles joined directly by an edge, or a subgraph of a triangle necklace.*

Proof. If two distinct triangles meet, subcubicity forces them to share an edge uv . Both u and v have degree three. Each of the other two vertices must have degree two: otherwise u would have two degree-three neighbors. All four vertices have their degrees exhausted, so the connected component is precisely $K_4 - e = N(2)$.

Otherwise the triangles are vertex-disjoint. No triangle has three degree-three vertices, since each would then have two degree-three neighbors. Choose a degree-two tip in each triangle and delete all chosen tips. Each deletion preserves connectivity because the two neighbors of the deleted vertex remain adjacent. Every original degree-three vertex loses a tip neighbor. Hence the resulting connected graph H has maximum degree two. If at least one triangle existed, H has at least two vertices and is a path or cycle. Each deleted tip corresponds to a marked edge of H , and these marked edges are pairwise vertex-disjoint.

Let ab and cd be consecutive marked edges in an order along H . Their connecting path has length at least one. If it is the single edge bc , then b and c have degree three in G . Saturation forces a and d to have degree two in G , hence degree one in H . They are the two endpoints of H , so the whole component is two triangles joined by one edge. In all other cases every gap between marked edges has length at least two. This also applies to the cyclic gap if H is a cycle.

If H is a cycle, G is already a necklace. If H is a path and the exceptional case does not occur, add a new path of length at least two between its endpoints. The closing gap includes any terminal tails and has length at least two. The augmented graph is a necklace containing G .

Finally, if G has no triangles then it has no degree-three vertices. It is a path, cycle, or isolated vertex. Embed it in a cycle C_m , $m \geq 3$, if necessary, and add a tip on any cycle edge. The result is $N(m - 1)$. This proves the lemma. $\square$

Packing chromatic number is monotone under taking subgraphs: deleting vertices or edges cannot shorten a distance. The exceptional barbell has a packing 4-coloring

$$(3, 2, 1) \quad \text{and} \quad (4, 2, 1)$$

on its two triangles, with the joining edge from the last vertex of the first triangle to the first vertex of the second. Thus it remains to color necklaces.

3 Coloring the tips and the backbone

Lemma 3 (sufficient backbone coloring). *A packing 5-coloring of the backbone in which both endpoints of every marked edge avoid color 1 extends to a packing 5-coloring of the necklace.*

Proof. Give every tip color 1. Tips are pairwise nonadjacent, and each has only the two non-1-colored endpoints of its marked edge as neighbors. Thus color 1 remains valid. Adding a tip supplies a length-two alternative to an existing backbone edge; it cannot shorten any distance between backbone vertices. The constraints on colors 2 through 5 are therefore preserved. $\square$

This is a sufficient construction. We do not assume that every packing coloring can be normalized to give all tips color 1.

4 A finite automaton for backbone colorings

Let W be the following set of length-five words:

$$W = \{s \in \{1, 2, 3, 4, 5\}^5 : s_i = s_j, i < j \implies j - i > s_i\}.$$

There are 390 such words. A state remembers the five most recently appended backbone colors. Define a Boolean relation B on W by

$$(s_1, s_2, s_3, s_4, s_5) B (s_2, s_3, s_4, s_5, c) \iff c \notin \{s_{6-c}, \dots, s_5\}.$$

Let A be the same relation restricted to transitions with $c \neq 1$. These transitions are exact: appending color c creates a conflict precisely when it already occurs within the preceding c positions. Earlier positions are irrelevant because $c \leq 5$.

All products below are Boolean relation products in traversal order:

$$(XY)(u, w) = 1 \iff \text{there exists } v \text{ with } X(u, v) = Y(v, w) = 1.$$

Let

$$S = \{s \in W : s_4 \neq 1, s_5 \neq 1\}, \quad |S| = 171.$$

Such a state may occur just after a marked edge. For a gap $g \geq 2$, put

$$R_g = (B^{g-1}AA) \Big|_{S \times S}.$$

This appends the $g - 1$ internal connector vertices followed by the two non-1-colored ports of the next triangle. It appends exactly $g + 1$ colors. For a word $w = (g_1, \dots, g_t)$, write $R_w = R_{g_1} \cdots R_{g_t}$.

Lemma 4 (cyclic closure). *If $L = \sum_i (g_i + 1) \geq 6$ and R_w contains a diagonal entry, then the backbone of $N(w)$ has the coloring required in Lemma 3.*

Proof. A diagonal entry gives a walk that starts and ends at the same five-color state. Repeat the walk in both directions. The resulting periodic color sequence obeys all distance restrictions on the infinite path, because every transition enforces all possible conflicts with earlier positions.

The appended string has period L . Assign it to the backbone cycle in cyclic order. If two distinct cycle vertices of color c violated the packing condition, a shortest arc between them would have length at most $c \leq 5$. That arc lifts to a segment in the repeated string and contradicts the path constraint. This includes arcs crossing the chosen cut in the cycle. The hypothesis $L \geq 6$ prevents a single vertex and its next periodic copy from being mistaken for two distinct vertices in this argument. Every marked edge has non-1 endpoints because its two vertices were appended by AA . $\square$

The only cases with $L < 6$ have $t = 1$ and $g = 2, 3, 4$. They are handled explicitly in Section 6.

5 The finite certificate

The following lemma is the computational part of the upper bound. Relations are compared entry by entry, so $X \subseteq Y$ means that every allowed transition of X is also an allowed transition of Y .

Lemma 5 (verified finite assertions). The automaton above has the following properties.

1. $B^{14} = B^{15}$ on all 390 states.
2. Each of the eighteen relations $M_i = R_{w_i}$ in Table 1 has a diagonal entry.
3. Every R_g , $5 \leq g \leq 15$, and every $R_g R_h$, $2 \leq g, h \leq 15$, contains at least one M_i .
4. For every $i \in \{1, \dots, 18\}$ and $g \in \{2, \dots, 15\}$, the product $M_i R_g$ contains at least one M_j .

Multiplication of the first equality by B , followed by induction, gives $B^m = B^{14}$ for every $m \geq 14$. Consequently

$$R_g = R_{15} \quad \text{for every } g \geq 15.$$

This is equality of boundary relations for all long connectors. It does not rely on any claim that subdividing edges preserves packing colorability.

i	w_i	A diagonal state	$g = 2$	$g = 3$	$g = 4$	$5 \leq g \leq 15$
1	22	23124	7	8	9	1
2	32	23124	10	11	12	1
3	23	21324	13	14	15	1
4	33	21324	16	17	18	1
5	24	12132	10	11	12	1
6	34	12132	1	3	5	1
7	222	23124	1	3	5	1
8	223	21324	1	3	5	1
9	224	12132	1	3	5	1
10	322	23124	1	3	5	1
11	323	21324	1	3	5	1
12	324	12132	1	3	5	1
13	232	23124	1	3	5	1
14	233	21324	1	3	5	1
15	234	12132	1	3	5	1
16	332	23124	1	3	5	1
17	333	21324	1	3	5	1
18	334	12132	1	3	5	1

Table 1. Certificate relations, diagonal witnesses, and right-extension inclusions. A word such as 223 denotes $(2, 2, 3)$, and a state such as 23124 denotes $(2, 3, 1, 2, 4)$. An entry j in column g certifies $M_j \subseteq M_i R_g$. The final column applies separately to each listed gap.

There are 207 seed inclusions and 252 extension inclusions. The self-contained program in Appendix A rebuilds the relations and checks all four assertions. It needs only the Python standard library and no saved search output. A separate implementation using explicit sets of successor states also passed these checks. Neither verification uses a SAT solver for the universal bound.

Proposition 6 (all gap sequences). *Every necklace $N(g_1, \dots, g_t)$ with backbone length at least six has a packing 5-coloring.*

Proof. First replace every $g_i \geq 15$ by 15 when computing its boundary relation; this leaves the product unchanged by Lemma 5. If $t = 1$, then its gap is at least five, so its relation contains some M_i by the seed assertion. If $t \geq 2$, the product of the first two gap relations contains some M_i . Suppose a prefix product P contains M_i . Appending a gap g gives

$$PR_g \supseteq M_i R_g \supseteq M_j$$

for some j , using monotonicity of composition and the extension assertion. Induction proves that the full product contains a certificate relation. Every certificate relation has a diagonal entry, and so does the full product. Lemmas 4 and 3 finish the proof. $\square$

6 Small cases and proof of the theorem

For $N(g)$, label the triangle $(0, 1, 2)$ with tip 2, and label its connector $1, 3, 4, \dots, g+1, 0$ in order. The following lists are valid colorings in vertex-label order:

g	colors on vertices $0, 1, \dots, g+1$
2	(3, 2, 1, 1)
3	(4, 3, 1, 2, 1)
4	(4, 3, 1, 1, 2, 1).

They can be checked directly; the separate verifier also checks all distances. Together with Proposition 6, these handle every necklace.

Proof of the upper bound in Theorem 1. Apply Lemma 2 to each connected component. Restrict a necklace coloring to each component that embeds in a necklace, and use the explicit barbell coloring for the exception. Subgraph monotonicity preserves validity. Colors may be reused in different components. $\square$

7 A sharpness example

Let $Q = N(2, 2, 2)$. Its triangles are $L_i R_i T_i$, for $i = 1, 2, 3$, and its connector vertices X_i satisfy $R_i X_i L_{i+1}$, cyclically. A packing 5-coloring is

i	$f(L_i)$	$f(R_i)$	$f(T_i)$	$f(X_i)$
1	5	2	1	1
2	4	2	1	1
3	3	2	1	1.

The graph has twelve vertices and satisfies the theorem’s hypotheses. An exhaustive search over all assignments of colors 1, 2, 3, 4, rejecting a partial assignment only when it already contains a distance violation, finds none. The complete finite check is included in Appendix A. A separate full-vertex SAT encoding was also checked UNSAT using both DRAT and LRAT proof checkers. Hence $\chi_\rho(Q) = 5$.

The shape of Q appears to coincide with the graph $\mathcal{G}_5$ in [1]. No novelty claim is made for this example. The proposed contribution is the universal upper bound and its reproducible certificate.

8 AI Disclosure

The brainstorming, computation, and construction of this proof was heavily assisted by GPT-6 Astra. We have manually reviewed all claims asserted and take full ownership of the work.

References and reading

1. A. El Zein and M. Mortada, *Impact of local girth on the S -packing coloring of k -saturated subcubic graphs*, arXiv:2603.25113v1, 26 March 2026. <https://arxiv.org/abs/2603.25113>. Read the definitions, Theorem 1, and Conjecture 2.
2. M. Mortada, A. El Zein and S. Al Hajjar, *On $(1,1,2,3)$ - and $(1,1,3,3,3)$ -Packing Colorings of Claw-Free Subcubic Graphs*, arXiv:2608.02566v1, 3 August 2026. <https://arxiv.org/abs/2608.02566>. This subsequent paper addresses different packing sequences and uses a triangle/diamond decomposition. Its stated results do not directly yield the ordinary five-color bound considered here. The decomposition itself should not be presented as a wholly new idea.
3. M. Mortada and O. Togni, *Further Results and Questions on S -Packing Coloring of Subcubic Graphs*, Discrete Mathematics 348(4) (2025), 114376. <https://arxiv.org/abs/2407.07424>. Background on saturation and related packing sequences.
4. H. Al Bazzal, *On S -packing colorings of subcubic graphs*, Australasian Journal of Combinatorics 92(2) (2025), 237–243. <https://arxiv.org/abs/2409.01769>. A concise example of a proof note in the area.

A Self-contained verification program

Save the listing below as `verify_standalone.py` and run it with ordinary Python 3. Do not use `-O`, which disables assertions. The program reconstructs the automaton, checks the finite certificate and its printed table entries, and verifies the sharpness example. It does not read any external data files or import any project modules.

```
"""Finite checks for the five-color theorem. Python 3; do not use -O."""
from itertools import product, combinations
import json

def good(s):
    return all(s[i] != s[j] or j-i > s[i]
               for i, j in combinations(range(len(s)), 2))

def mul(a, b):
    result, cache = [], {}
    for row in a:
        if row not in cache:
            bits, out = row, 0
            while bits:
                low = bits & -bits
                out |= b[low.bit_length()-1]
                bits -= low
            cache[row] = out
        result.append(cache[row])
    return tuple(result)

def sub(a, b):
    return all(x & ~y == 0 for x, y in zip(a, b))

W = [s for s in product(range(1, 6), repeat=5) if good(s)]
wi = {s: i for i, s in enumerate(W)}
```

```

def transition(palette):
    return tuple(sum(1 << wi[s[1:]+(c,)] for c in palette
                    if good(s+(c,))) for s in W)
B, A = transition(range(1, 6)), transition(range(2, 6))
powers = [tuple(1 << i for i in range(len(W)))]
for _ in range(15):
    powers.append(mul(powers[-1], B))
assert powers[14] == powers[15]
ids = [i for i, s in enumerate(W) if s[-2] != 1 and s[-1] != 1]
S = [W[i] for i in ids]
assert (len(W), len(S)) == (390, 171)
R = {}
for g in range(2, 16):
    full = mul(mul(powers[g-1], A), A)
    R[g] = tuple(sum(1 << j for j, v in enumerate(ids)
                    if full[u] & (1 << v)) for u in ids)
words = ['22', '32', '23', '33', '24', '34', '222', '223', '224',
         '322', '323', '324', '232', '233', '234', '332', '333', '334']
M = []
for word in words:
    m = tuple(1 << i for i in range(len(S)))
    for letter in word:
        m = mul(m, R[int(letter)])
    M.append(m)
diag = ['23124', '23124', '21324', '21324', '12132', '12132',
        '23124', '21324', '12132', '23124', '21324', '12132',
        '23124', '21324', '12132', '23124', '21324', '12132']
for m, word in zip(M, diag):
    i = S.index(tuple(map(int, word)))
    assert m[i] & (1 << i)
def covered(m):
    return any(sub(a, m) for a in M)
seeds = 0
for g in R:
    if g >= 5:
        assert covered(R[g])
        seeds += 1
    for h in R:
        assert covered(mul(R[g], R[h]))
        seeds += 1
table = [[7,8,9], [10,11,12], [13,14,15], [16,17,18],
         [10,11,12]] + [[1,3,5]]*13
extensions = 0
for i, m in enumerate(M):
    for g in R:
        j = table[i][g-2] if g <= 4 else 1
        assert sub(M[j-1], mul(m, R[g]))
        extensions += 1
assert (seeds, extensions) == (207, 252)

def necklace(gaps):
    t = len(gaps)
    adj = [set() for _ in range(3*t)]
    def edge(u, v):
        adj[u].add(v)
        adj[v].add(u)
    for i, g in enumerate(gaps):
        for u, v in combinations(range(3*i, 3*i+3), 2):

```

```

        edge(u, v)
    old = 3*i+1
    for _ in range(g-1):
        new = len(adj)
        adj.append(set())
        edge(old, new)
        old = new
    edge(old, 3*((i+1) % t))
    return adj

def checked_distances(adj, colors):
    n = len(adj)
    d = [[0 if u == v else 1 if v in adj[u] else n+1
          for v in range(n)] for u in range(n)]
    for u, ns in enumerate(adj):
        assert len(ns) <= 3 and all(u in adj[v] for v in ns)
        if len(ns) == 3:
            assert sum(len(adj[v]) == 3 for v in ns) <= 1
            assert any(v in adj[w] for v, w in combinations(ns, 2))
    for k in range(n):
        for u in range(n):
            for v in range(n):
                d[u][v] = min(d[u][v], d[u][k]+d[k][v])
    assert len(colors) == n and all(1 <= c <= 5 for c in colors)
    assert all(colors[u] != colors[v] or d[u][v] > colors[u]
               for u, v in combinations(range(n), 2))
    return d
for g, cs in [(2, [3,2,1,1]), (3, [4,3,1,2,1]),
              (4, [4,3,1,1,2,1])]:
    checked_distances(necklace([g]), cs)
barbell = [{1,2},{0,2},{0,1,3},{2,4,5},{3,5},{3,4}]
checked_distances(barbell, [3,2,1,4,2,1])
Q = necklace([2,2,2])
d = checked_distances(Q, [5,2,1,4,2,1,3,2,1,1,1,1])
order = sorted(range(12), key=lambda v: (-len(Q[v]), v))
assignment, calls = [0]*12, 0
def search(depth):
    global calls
    calls += 1
    if depth == 12:
        return True
    u = order[depth]
    for c in range(1, 5):
        if all(assignment[v] != c or d[u][v] > c
              for v in order[:depth]):
            assignment[u] = c
            if search(depth+1):
                return True
            assignment[u] = 0
    return False
assert not search(0)
print(json.dumps({'status':'PASS', 'states':len(W), 'boundary':len(S),
                  'relations':len(M), 'seeds':seeds,
                  'extensions':extensions, 'sharpness_nodes':calls}))

```